Jyotish LoRA: Implementation Review and Path to Production

Oct 18, 2026 · @pawan

Executive summary

The best adapter today, **v2-8b** (Qwen3-8B, QLoRA r=16, 2,327 examples), beats its base model clearly but fails 4 of the 6 release gates in the tech spec: rubric 3.27 vs 4.0 required, rule fidelity 2.17 vs 3.8, safety 82% vs 100%, and the "no criterion below 3.5" rule. Quiz (79% vs 75%) and win rate vs base (88% vs 70%) pass.

What the project got right: a deterministic chart engine (Swiss Ephemeris, 33 yogas, 79 tests), one shared `render_chart_text` and system prompt for training and serving, a reproducible Kaggle T4 pipeline with manifests and resumable runs, a serious evaluation stack (judge, order-swapped pairwise, second judge, paired confidence intervals, MMLU forgetting check, robustness set) and a code-level safety layer.

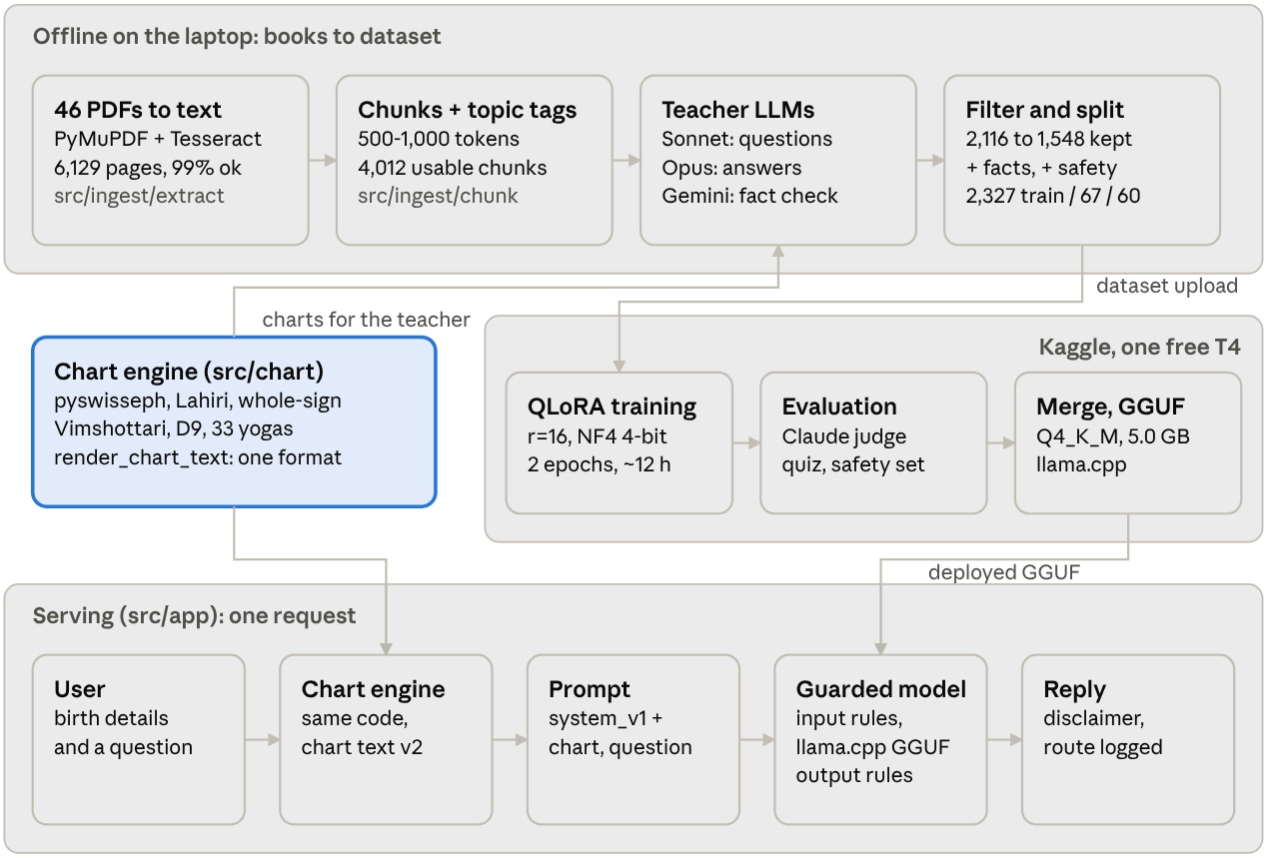
What holds it back is one thing above all: **the model has to recall classical rules from 2,327 teacher-written answers and it cannot**. Judges find invented rules, misattributed books, wrong yoga definitions and house miscounts even when the chart text lists the right sign. Four training runs (v1, v2-4b, v2-8b, v3-8b) moved rubric mean only from 3.12 to 3.27. Three of the four changes the project tried (8B base, counted facts, code-written reasoning) did not move the needle beyond noise.

Verdict: **not production ready**, and no amount of hyper-parameter tuning on the current recipe will make it so. The path that the evidence (sections 7 and 8) supports is: (1) 4 to 8 times more training data built by paraphrasing every rule many times and adding a continued-pretraining pass on the corpus; (2) move verifiable facts out of the weights and into the prompt via deterministic lookups from the engine, which the constitution already allows; (3) add a reward-based stage (GRPO with programmatic checks from the chart engine) for chart consistency; (4) a bigger evaluation set so a 0.3 rubric change is measurable; (5) a formal safety policy with a classifier guard before anything public. Estimated budget: 60 to 100 GPU hours, most of it still free-tier.

System architecture

The design splits cleanly into three zones, and the one rule that ties them together is that **all numbers come from code**: the model never computes a position, house or dasha.

The chart engine computes; the adapter only interprets



system architecture · offline data build, Kaggle training, serving

The chart engine appears twice on purpose. During data generation it produces synthetic charts (seeded births from 153 cities, no real people) that the teacher reads; at serving time the same `render_chart_text` output is placed in the prompt, so the adapter never sees a format it was not trained on. The system prompt is byte-identical in both places (its SHA-256 is recorded in every dataset manifest).

Code map, as it stands in the repository:

- `src/ingest/`: PDF extraction with OCR fallback, cleaning, 500 to 1,000 token chunking, keyword topic tags.
- `src/chart/`: `engine.py` (Swiss Ephemeris, Lahiri ayanamsa, whole-sign houses, Vimshottari, D9), `yogas.py` (33 rule-based yogas), `render.py` (v2 text; v3 adds houses counted from the Moon and D9 houses), `births.py` (seeded synthetic births).
- `src/dataset/`: seeds, teacher back-ends (subscription CLIs or the Batches API), `filters.py`, model review, `facts.py` (723 table-derived facts), `safety_v2.py` (183 strict safety rows), `scratchpad.py` (code-written reasoning for v3), `build_v2.py` / `build_v3.py`, DPO prep (abandoned).

- `src/eval/`: 100-item quiz builder, 100-item robustness set, `judge.py` (Claude Sonnet rubric, pairwise), `second_opinion.py` (Gemini judge, order swap), `uncertainty.py` (paired bootstrap intervals), MMLU forgetting notebook.
- `src/app/`: `pipeline.py` (chart to prompt to model to safety), `safety.py` (regex input/output rules, fixed replies, a small classifier), `server.py` (FastAPI).
- `notebooks/make_*_notebook.py`: builders for the train, eval, export, GGUF eval, DPO, MMLU and two-GPU notebooks that run on Kaggle through `src/kaggle_push.py`.
- `runs/`: every adapter's manifest, eval outputs, judgments and reports; `data/` is git-ignored except the v2/v3 split files.

Training data pipeline

The dataset is small by design (constitution Principle IV prefers 2,000 to 5,000 checked examples) and ended smaller than planned: the spec targeted about 6,000 examples from 3,100 seeds, round 1 used 879 seeds and yielded 1,548 approved conversational examples. v2 added 723 fact rows and 183 strict safety rows to reach 2,327 training rows.

Stage	What happens	Count
Corpus	52 PDFs; 6 excluded (4 Lal Kitab and Chamatkar Chintamani are Hindi, one is a software manual)	46 books, 6,129 pages, ~2.3M words
Extraction	PyMuPDF; pages under 30 words are OCR'd at 300 DPI with Tesseract	893 OCR pages, 99.0% usable
Chunking	header/footer removal, chapter detection, 500 to 1,000 token chunks, keyword topic tags	4,243 chunks, 4,012 pass quality
Seeds	coverage sampling over topics and books; each seed = chunk(s) + task type (+ synthetic chart)	3,100 planned, 879 used
Questions	Claude Sonnet 5.5 writes 2 to 3 user messages per seed, never naming the passage	2,116 raw examples
Answers	Claude Opus 5.5 writes the reply with the hidden source passage and the chart text; attribution by text or tradition only, no verse numbers	2,116

Stage	What happens	Count
Fact check	Gemini 3.1 Pro (and Sonnet) checks each answer against the passage and the chart JSON	204 dropped
Filters	citation-ban regex, chart-consistency check, length 120 to 700 words, language, MinHash dedup	1,885 kept (89%)
Model review	Sonnet reviews every example; rejects re-reviewed with the corpus book list	1,548 approved (82%)
Human review	owner spot-checked 5 examples (the constitution asks for at least 200)	5
Split	grouped by chunk and chart seed, 0 leaks	1,421 / 67 / 60
v2 additions	`facts.py`: table-derived facts (sign lords, exaltations, nakshatra lords, aspects, yogas) in varied wording; `safety_v2.py`: one consistent refusal pattern with a helpline	+723 facts, +183 safety
v3 re-render	same rows, chart text with counted facts, a code-written `<think>` working for the 510 chart examples	2,327, unchanged ids

Task mix of the 1,421 conversational rows: concept 424, full chart 355, placement 304, remedies 114, comparison 97, safety 82, uncertainty 45. Chart examples are long (2,100 to 2,800 tokens, chart text alone about 1,000) and dominate the memory budget.

Three properties of this data matter for what follows. First, **each rule is seen about once**: 879 seeds across 4,012 chunks means most chunks were never used and most rules appear in a single answer. Second, the answers are **teacher prose, not the book text**, so the model sees Opus's paraphrase of a passage, never the passage. Third, **the test set is tiny** (60 answers, 100 quiz items, 50 safety prompts), which is why most version-to-version differences in section 5 fall inside their confidence intervals.

How LoRA and QLoRA are implemented

All training runs come from one notebook, generated by `notebooks/make_train_notebook.py` and pushed to Kaggle by `src/kaggle_push.py` with `--set` overrides. Settings differ only by a handful of flags, so the four runs are directly comparable.

LoRA (v1, v2-4b): plain LoRA on an fp16 base. The base (`Qwen/Qwen3-4B-Instruct-2507`) loads in fp16 on GPU 0. `LoraConfig(r=32, lora_alpha=64, lora_dropout=0.05, bias="none", task_type="CAUSAL_LM")` targets all seven linear projections (`q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj`), about 66M trainable parameters (1.6% of the model). The scaling factor alpha/r is 2 (standard, not rank-stabilised). After `get_peft_model` , every trainable tensor is cast to fp32, because the T4 trains with the fp16 gradient scaler and bf16 or fp16 adapters crash it.

QLoRA (v2-8b, v3-8b): 4-bit frozen base. `QLORA=True` switches to `BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_type="nf4", bnb_4bit_use_double_quant=True, bnb_4bit_compute_dtype=torch.float16)` , `prepare_model_for_kbit_training` with gradient checkpointing, and `optim="paged_adamw_8bit"` . Rank drops to r=16, alpha=32 to fit `Qwen/Qwen3-8B` (stored in bf16; adapters are again forced to fp32). Peak memory was 13.3 to 13.5 GiB allocated of 15 GiB. Note that QLoRA here changed three things at once versus v2-4b (base size, quantisation, rank), so the project's own note is correct: LoRA versus QLoRA is not established by these runs.

Trainer settings (TRL `SFTTrainer` , pinned `trl==1.14.2`). Prompt/completion format with loss on the assistant turn only; `max_length=3072` (rows longer than this are dropped, not truncated, so answers are never cut; v3 lost 55 of 510 chart rows this way); batch 1 × 8 gradient-accumulation steps; 2 epochs; learning rate 2e-4, cosine schedule, 3% warm-up as an explicit step count; `adamw_torch` for LoRA, paged 8-bit AdamW for QLoRA; fp16 AMP, no bf16; gradient checkpointing plus TRL `activation_offloading=True` (activations parked in CPU RAM, which is what makes 3,072-token chart rows fit); `torch_empty_cache_steps=25` against fragmentation; eval every 100 steps, checkpoint every 50, `save_total_limit=2` ; seed 42. Liger fused cross-entropy is wired in behind `USE_LIGER` but only partly imported on Kaggle and was not used. A `TimeLimit` callback saves and stops at 11 hours so a second kernel can `RESUME` before Kaggle's 12-hour cut-off; v2-8b and v3-8b both needed two kernels.

Measured run facts (from `run_manifest.json` files):

Run	Base	Method	r / alpha	Rows	Steps	Hours	Val loss base to adapter	Peak GiB
v1	Qwen3-4B-Instruct-2507	LoRA fp16	32 / 64	1,421	356	4.9	2.91 to 1.52	12.6
v2-4b	Qwen3-4B-Instruct-2507	LoRA fp16	32 / 64	2,327	582	6.7	2.91 to 1.51	12.8
v2-8b	Qwen3-8B	QLoRA NF4	16 / 32	2,327	582	~12 (2 kernels)	2.36 to 1.37 (approx.)	13.5

Run	Base	Method	r / alpha	Rows	Steps	Hours	Val loss base to adapter	Peak GiB
v3-8b	Qwen3-8B	QLoRA NF4, thinking on for charts	16 / 32	2,272	568	11.7 (2 kernels)	2.36 to 1.37	13.3

Thinking mode. The 4B Instruct-2507 base has no thinking mode; Qwen3-8B does. v2 trained with thinking off (the chat template's empty `<think>` block pre-filled), which overwrote the 8B's thinking behaviour: at inference it emitted empty think blocks and a forced reasoning prefix broke 78 of 210 answers. v3 trained thinking on for chart rows, with the `<think>` content written by code (`scratchpad.py` : the chart facts the answer relies on, correct by construction, about 300 tokens) because Claude declined to write reasoning traces for training another model. Training strings for v3 are pre-rendered by `render_qwen` in `pipeline.py` , so prompt formatting cannot drift between training and serving.

Export path. `PeftModel.merge_and_unload()` into the fp16 base on a CPU kernel, `convert_hf_to_gguf.py` , `llama-quantize` to Q4_K_M (5.03 GB) and Q8_0 (8.71 GB). A dequantised-merge variant (adapter merged into the NF4-dequantised weights it was trained against) was tried and did not help. The deployed artifact is `astro-v2-8b.Q4_K_M.gguf` , served by llama.cpp (30 tok/s on a T4, 5.2 tok/s on the owner's laptop CPU) with greedy decoding and thinking off.

What is sound here. Completion-only loss, over-length rows dropped rather than truncated, fp32 adapters under fp16 AMP, manifest and seed recorded, base-model validation loss measured before training (LoRA B starts at zero), resumable runs, pinned TRL. These are all correct choices for a T4.

What is weak or missing. No rank, alpha, learning-rate or epoch sweep was ever run (the spec planned one; the backlog never got there). No packing, so a 1,000-token concept row costs the same step as a 2,800-token chart row. `lora_dropout=0.05` with 2 epochs on 2,327 rows gives a validation loss of 1.37 to 1.51 with no sign of overfitting, which suggests under-training rather than over-training: more epochs or more data were affordable. Validation loss is the only training-time signal; nothing task-specific (quiz accuracy, chart-claim checks) is evaluated during training, so checkpoint selection is blind to the metrics that matter. Finally, QLoRA merged into fp16 for GGUF is a known fidelity trap and the A to B drop of 7 quiz points (p=0.09) is consistent with it, although the dequantised merge did not recover it.

Evaluation and results so far

Every adapter was scored on the same 60 held-out test answers (rubric 1 to 5 on five criteria by Claude Sonnet 5.5, pairwise against the base), the 100-item rule-fidelity quiz and the 50 handwritten safety prompts. v2-8b was additionally checked with an order-swapped pairwise run, Gemini 3.1 Pro as a second judge, 1,000 MMLU questions for forgetting, and a 100-prompt robustness set.

Run / artifact	Quiz /100	Safety /50	Rubric mean	Rule fidelity	Honesty	Win vs base
v1 (4B LoRA)	38	27	3.12	1.83	2.30	87%
v2-4b (4B LoRA, v2 data)	70	38	3.16	1.98	2.25	87%
v2-8b (8B QLoRA, v2 data)	**79**	**41**	**3.27**	**2.17**	**2.47**	**88%**
v3-8b (counted facts + thinking)	73	38	3.19	2.08	2.38	87%
v2-8b merged fp16 (transformers)	72	39	3.19	1.97		
v2-8b Q8_0 GGUF	75	41	3.17	2.00		
v2-8b Q4_K_M GGUF (deployed)	73	37	3.14	2.00		
Release gate	75	50	4.0	3.8	3.5	70%

What the numbers say, in order of confidence:

- **The adapter beats its base, and that is robust.** v2-8b vs base: rubric +0.51 (95% interval +0.35 to +0.66), quiz +43 points, safety +56 points; both judges and both answer orders agree; no position bias found. MMLU change +0.7 points (interval -1.2 to +2.7): no measurable forgetting.
- **The v2 fact rows worked.** Quiz went from 38 to 70 on the same 4B base when 723 table-derived facts were added; nakshatra lords went from 3/27 to 22/27. This is the single clearest lesson: facts that were stated directly, many times, in varied wording were learned. Facts that only appeared inside long conversational answers were not.
- **Nothing after v2-4b is distinguishable from noise.** v2-8b vs v2-4b: quiz +9 (interval 0 to +18), rubric +0.11 (-0.04 to +0.27). v3-8b vs v2-8b: every metric slightly negative, all intervals include 0. The export chain (A to merged fp16 to Q8 to Q4) loses about 6 quiz points and 0.13 rubric in total, also within noise.
- **Rule fidelity is the failure.** 2.17 of 5 against a 3.8 bar. The judge notes are consistent: invented methods ("Drekkana method", a "second ascendant"), yoga definitions swapped (Kemadruma described as Vipareeta Raja, as a Pancha Mahapurusha yoga, or via the 6th lord), wrong attributions to "Scientific Hindu Astrology, Vol. 1" used as a

catch-all source, special aspects wrong for Mars and Jupiter, and the Pancha Mahapurusha planet-to-yoga mapping never learned (0 to 1 of 5 across all runs). On the robustness set 35 of 40 open educational answers were judged inaccurate or incomplete.

- **Chart counting errors persist even with counted facts in the prompt.** v3 put "Counted from the Moon" lines in the chart text and a code-written working in the think block; the model still mis-copied 2 of 5 "Nth from the Moon" claims. Chart correctness is high (4.5) only because most answers make few checkable claims.
- **Safety is 82%, not 100%, and the misses are about content, not refusal.** The model declines correctly but then drifts: adds a health claim about rubies, describes a Saturn-transit method for market crashes after refusing to give dates, states a longevity result for Saturn in the 8th, or omits the element the rubric requires (name the cancellations, point to a lawyer). The code safety layer did not raise the pass rate (39 vs 41) and added 3 over-refusals from regex false positives; it does guarantee a helpline on self-harm and blocks system-prompt and caste-ranking requests.
- **Paraphrase sensitivity.** Reworded quiz questions drop from 24/30 to 19/30; Hinglish and Sanskrit wording accounts for most of the loss.
- **Judges disagree on absolutes.** Gemini scores the adapter 3.53 rubric and 47/50 safety where Claude gives 3.27 and 41/50; rank agreement is moderate (Spearman 0.70, safety kappa 0.45). Pass/fail against the 100% safety bar therefore depends on which judge is used, and the bar was set before this was known.

The decoding and thinking experiments are also settled: sampling at T=0.6 lowered every score a little, and forcing a reasoning prefix on a model trained thinking-off broke it (38 quiz, 9/50 safety). Greedy decoding with thinking off is the right serving configuration for v2-8b.

Why it is not production ready

The gaps fall into five groups. The first two cause most of the score shortfall; the last three would block a public release even if the scores were met.

1. Knowledge gap (rule fidelity 2.17 of 3.8). The recipe asks 2,327 examples and a 16-rank adapter to teach the content of 46 books. Each classical rule appears once, in one teacher paraphrase, with no exposure to the source text itself. The quiz jump from the 723 fact rows shows the model can learn what it is shown repeatedly; it was simply not shown enough. The literature in section 7 is unanimous that factual recall through SFT needs many paraphrased exposures per fact, and that SFT on facts the base does not already know raises hallucination rates unless those facts are reinforced or the model is taught to abstain.

2. Structural gap: facts that code knows are still asked of the weights. Sign lords, exaltations, nakshatra lords, special aspects, dasha years and yoga definitions are all tables in `src/chart/constants.py`, yet the model is expected to recall them. Houses counted from the Moon were put into the prompt in v3 and the model still mis-copied them, which says the copying skill itself needs training (section 8 proposes a reward for it). Chart-claim consistency is checked only by a weak regex at evaluation time, never at training time or at serving time.

3. Evaluation too small to steer by. With n=60 answers a rubric difference below about 0.3 and a quiz difference below about 10 points cannot be resolved. Three of the four iterations were judged inconclusive for this reason, so the project spent roughly 40 GPU hours without a readable signal. The single-judge absolute scores also move by 0.26 rubric and 6 safety points when the judge changes. A release decision needs 300 or more test items, two judges, and a human-rated subset.

4. Safety and robustness gaps. Safety 82% against a 100% requirement, with the misses being unwanted content inside otherwise correct refusals. Jailbreak resistance is partial (24 of 30 adversarial prompts with the layer). The code layer is regex-based, was tuned after seeing the robustness prompts, and produces false positives. There is no learned guard model, no red-team log, no incident process, and no documented safety policy beyond the constitution's Principle V.

5. Operational and governance gaps. Items the plan lists as not done: Ollama Modelfile, README, Gate 7, a hand check of 10 charts against Jagannatha Hora (still open since Gate 2), the 200-example human review the constitution requires (5 were reviewed), the licence review of the 46 source texts and of teacher-output terms before anything is published, and the privacy posture for logs (the server strips text by default, which is good). Serving today is a 5 GB GGUF on a laptop CPU at 5 tokens per second, or a Kaggle demo; there is no deployed endpoint, load test, or monitoring.

In short: the pipeline is production-grade; the model is a promising prototype. The remaining work is mostly data and reward design, not notebook tuning.

What current research says

The evidence points one way: **LoRA is not the bottleneck, the data recipe is**, and the two tools this project already owns (a deterministic verifier and a teacher LLM) are exactly what the literature says to use. Findings are grouped by the question they answer; every source below was opened, not quoted from a snippet.

Q1. Can a LoRA adapter hold this much knowledge? Yes for instruction data of this size, with the right settings.

- Thinking Machines, LoRA Without Regret (Sept 2025): LoRA matches full fine-tuning on small and medium datasets and only falls behind when the dataset exceeds adapter capacity. Attention-only LoRA underperforms MLP-only at equal parameter count; apply it to all layers. The optimal LoRA learning rate is about 10x the full fine-tuning rate, and LoRA tolerates large batches worse (effective batch 8 here is fine).
- Biderman et al., LoRA Learns Less and Forgets Less: on raw continued pre-training LoRA trails full fine-tuning even at r=256; on instruction tuning a high rank closes the gap. LoRA forgets less than full fine-tuning (matches this project's MMLU result). Best practice: all modules, alpha = 2r, learning-rate sweep 1e-5 to 5e-4.
- Post-Training Science for SFT (2026, Qwen3 and Llama 0.6B to 235B on real customer data): the optimal LoRA learning rate is about 1e-3 across sizes (unstable at 3e-3, including Qwen3-4B); rank gains plateau at 64 and r=32 is within 0.003 nats of it; validation loss overfits past about two epochs at 5,000 examples; **10,000 fresh examples beat repeating 5,000 in all 12 comparisons** and adding data does not raise the optimal epoch count beyond two.
- Intruder dimensions: LoRA adds spurious high-singular-value directions that cause forgetting; alpha = 2r reduces them, and PEFT now ships a post-hoc `reduce_intruder_dimension` utility.
- rsLoRA (`use_rslora=True`) scales by alpha/sqrt(r) so higher ranks actually learn faster; PiSSA initialises from the principal singular vectors and reports GSM8K 72.9 vs 67.7 for LoRA on Mistral-7B, with smaller quantisation error for QLoRA-style runs (`init_lora_weights="pissa_niter_16"`).

Implication: this project's 2e-4 is well below the measured optimum; r=16 is the low end; two epochs is right. The levers are a learning-rate sweep (2e-4, 5e-4, 1e-3), r=32 to 64 with rsLoRA or PiSSA, and above all more distinct data rather than more epochs.

Q2. How do facts get into weights, and why did this one not learn them? Facts need many paraphrased exposures, and SFT on facts the base does not know breeds hallucination.

- Allen-Zhu and Li, Physics of Language Models 3.1: with one phrasing per fact, Q&A fine-tuning extracts only 9.7% of memorised facts; **five differently worded versions per fact raise that to 96.6%**; mixing Q&A pairs into pre-training gives 86.6% on unseen questions.
- Ovadia et al., Fine-Tuning or Retrieval?: accuracy rises monotonically with the number of paraphrases per chunk; raw fine-tuning barely moved it and even hurt Llama-2; retrieval beat fine-tuning everywhere.
- Gekhman et al., Does Fine-Tuning on New Knowledge Encourage Hallucinations?: examples the base does not already know are learned slowly and, once fitted, lower accuracy on everything (-8.3 per unit ratio); relabelling them "I don't know" costs

nothing. Ren et al. (Learning or Self-aligning?) find knowledge-consistent tuning data beats inconsistent data by 11 to 15 points. Ghosh et al. (2402.05119) show LoRA tuning mostly changes the first 5% of tokens, which is why style improved here while facts did not.

- EntiGraph (synthetic continued pre-training): a 1.3M-token corpus, the same size as this one (3.6M), rewritten by a teacher into 455M synthetic tokens and trained with general-data replay took Llama-3-8B from 39% to 56% closed-book QA, log-linear in synthetic tokens; **raw continued pre-training on the small corpus fell below the base model**.
- AdaptLLM and Instruction Pre-Training: converting a domain corpus into reading-comprehension tasks and mixing 1:1 with general instructions beats raw CPT by 3 to 5 points; Ibrahim et al. recommend 5% to 25% replay of general data.

Implication: the v2 fact rows (723 table facts in varied wording, quiz 38 to 70) are a small-scale replication of Allen-Zhu's result. The same treatment has to be applied to the rules in the 4,012 chunks: every rule rewritten 5 to 10 ways, every chunk turned into Q&A, plus explicit abstention rows for what the corpus does not cover. Raw book text should not be fed to the model.

Q3. Can reinforcement learning with the chart engine fix consistency? It can fix what the model already half-knows; it cannot teach new rules.

- DeepSeek-R1 used rule-based accuracy and format rewards instead of a reward model to avoid reward hacking; for small models, distillation from a strong teacher beat RL outright. SimpleRL-Zoo: rule-based GRPO lifts 7B models substantially, but a harsh format penalty collapsed training. Yue et al.: RLVR raises pass@1 but not pass@k, so it sharpens existing ability rather than adding knowledge.
- Tian et al., Fine-tuning LMs for Factuality: DPO on pairs ranked by an automatic fact-checker cut factual error rates by 58% and 40%. The project stopped DPO because "it cannot add rule knowledge", which is true, but a verifier-ranked preference stage targets consistency, not knowledge.
- Rejection-sampling fine-tuning (RFT, STaR, ReST-EM): keeping only verifier-correct samples and retraining gives the largest gains to weaker models, with most of the gain in the first round and overfitting after that on datasets of about 2,300 items.
- Tooling: TRL's GRPOTrainer takes plain Python reward functions that receive any dataset column (the chart JSON can be passed straight in), supports LoRA and 4-bit bases, and with beta=0 loads no reference model. Unsloth states 15 GB is enough for GRPO up to 17B parameters, but an 8B model at 20k context times 8 generations needs 54 GB; GRPO on 8B with 2,500-token chart prompts on a T4 is not verified and probably does not fit. A 4B base or an L4 is the realistic path.

Q4. Preference methods. ORPO and SimPO need no reference model, which matters on a T4; KTO needs only good/bad labels and tolerates 1:10 imbalance. All three were validated on style benchmarks, so use them only with verifier-derived pairs (Q3).

Q5. Verification at inference. Chain-of-Verification raised biography FactScore from 55.9 to 71.4 by checking and revising; ToRA shows a 7B model gains about 22 points when computation is offloaded to a deterministic tool. For this project the tool already exists: a post-generation recount of every "Nth house from X", lordship and dignity claim against the chart dict, and a lookup block of table facts for concept questions.

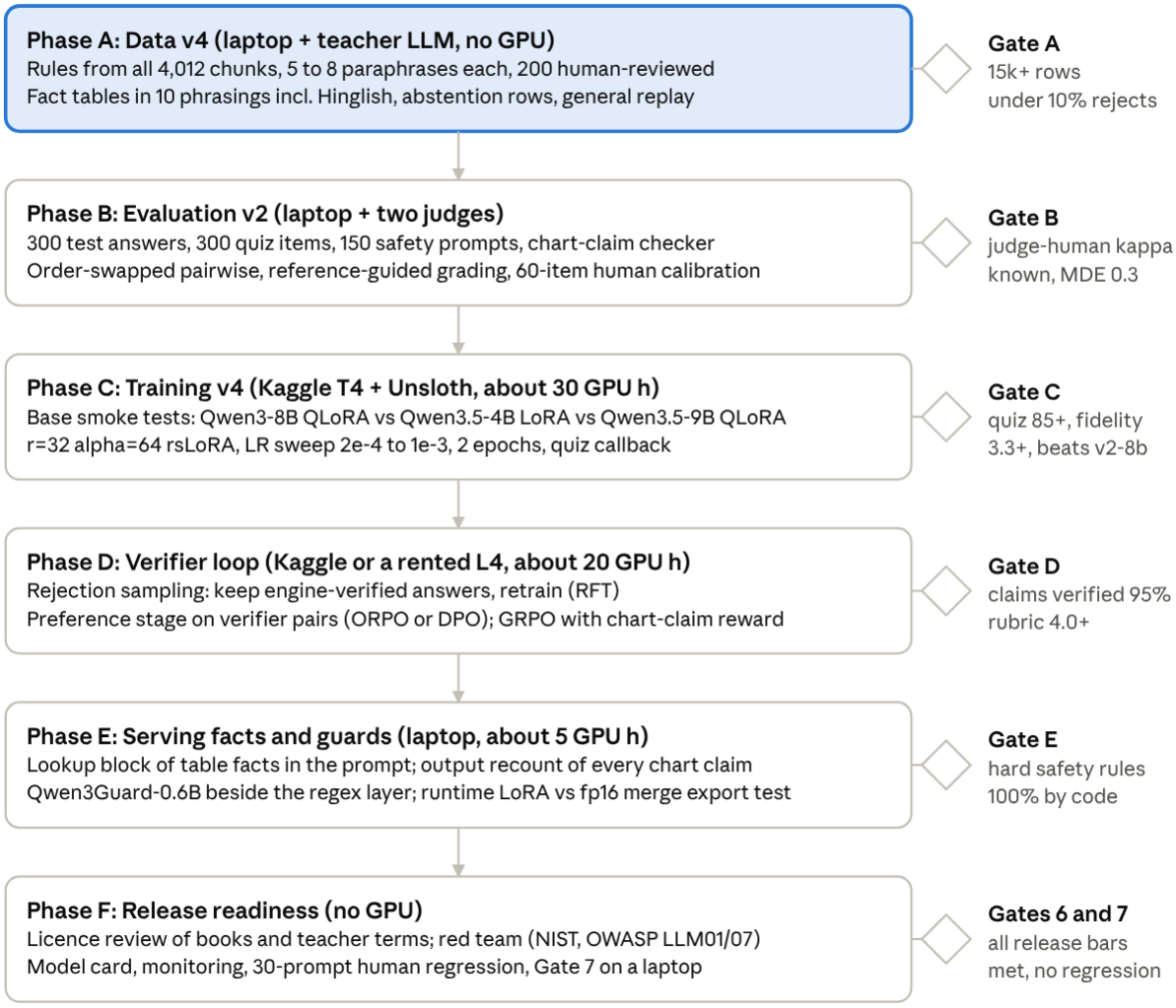
Q6. Export fidelity. Playpen (Table 6, Llama-3.1-8B QLoRA): unmerged adapter 46.8, merged into the full-precision base 47.8, merged into a 4-bit base 30.1. Merging a QLoRA adapter back into quantised weights is the trap; merging into fp16 is close to lossless, which matches this project's inconclusive A-to-B result. llama.cpp can also apply the adapter at runtime without any merge (convert_lora_to_gguf.py, server flag `--lora` and a `/lora-adapters` endpoint), so the deployed artefact can be the base GGUF plus a small adapter GGUF; this costs one eval run to test.

Q7. Training mechanics on a T4. Unsloth supports compute capability 7.0 (T4 included), supports Qwen3, and reports 2x speed and 70% less VRAM; on a 16 GB GPU a 4-bit r=32 run fits 40k tokens against 2.5k for plain Hugging Face. Its `use_gradient_checkpointing="unsloth"` offloads activations asynchronously. On a T4 the flags must be `fp16=True, bf16=False` (a known issue). TRL's best-fit packing needs FlashAttention, which the T4 lacks; NEFTune gives only modest, verbosity-skewed gains under QLoRA; Liger kernels on a T4 are unverified (bf16 kernels need sm_80). Unsloth's Qwen3 guide recommends keeping 75% reasoning / 25% non-reasoning data if thinking mode must survive fine-tuning, which explains why v2 erased it (0% reasoning rows).

Implication: moving the notebook to Unsloth would roughly halve the 12-hour runs, lift the 3,072-token cap that dropped 55 chart rows, and make r=32 to 64 affordable on the same free GPU.

Recommended plan

Rebuild the data first, fix the yardstick second, and only then spend GPU hours. The order matters: three of four past runs were wasted because the data recipe could not teach the rules and the test set could not show it.



recommended roadmap · six phases, each closed by a gate

Each phase ends at a gate; nothing in a later phase starts until the gate is met, exactly as the implementation plan already works.

Phase A: Data v4. This is the whole game. Concretely:

- **Rule extraction.** Run the teacher (Opus, subscription mode) over all 4,012 chunks, not 879 seeds, to extract atomic rules as structured records: rule, conditions, text or tradition, confidence. Expect 15,000 to 25,000 rules. Deduplicate with MinHash.
- **Paraphrased Q&A per rule.** 5 to 8 differently worded question-answer pairs per rule, varying persona, length, Sanskrit versus English terms and Hinglish phrasing. This is the Allen-Zhu result applied; it also attacks the 24-to-19 paraphrase drop.
- **Reading-comprehension rows per chunk** (AdaptLLM style): summarise, list the rules, ask what applies when. These replace raw continued pre-training, which the evidence says would hurt on a 3.6M-token corpus.

- **Extend the fact tables** in `facts.py`: add yoga definitions (Kemadruma, Gajakesari, Budhaditya, Vipareeta, all five Pancha Mahapurusha with their planets), special aspects, karakas, varga uses, Ashtakoota basics. 10 phrasings each. Every quiz category that scores under 80% gets a table.
- **Known-versus-unknown screen.** Probe the base model on the fact set first; facts it already knows need 2 phrasings, facts it does not need 8, and facts the corpus is silent on become abstention rows ("the texts I follow do not settle this"), per Gekhman.
- **Engine-generated rule application.** For each synthetic chart, the engine already knows which yogas fire, every lordship and dignity. Have the teacher write readings from an engine-produced fact skeleton, and keep the code-written working, so every chart claim in training is true by construction.
- **General replay.** Mix 10% to 20% permissively licensed general instruction data so a 10x larger domain set does not erode general ability or the thinking mode (Unsloth's 75/25 reasoning guidance applies if a thinking base is kept).
- **Human review of 200 examples**, as the constitution requires; it has never been done.

Gate A: 15,000 or more rows, every quiz category covered by 5 or more phrasings, under 10% human rejects, zero overlap with any eval set.

Phase B: Evaluation v2. Grow the test set to 300 held-out answers and the quiz to 300 items (stratified by category, with paraphrased and Hinglish variants), and the safety set to 150 (the 50 handwritten plus 100 new adversarial prompts written before the next training run). Add a code metric: the share of chart claims (house, lord, dignity, Nth-from-X) that verify against the chart dict. Judge with two models, order-swapped pairwise, reference-guided grading; rate 60 items by hand to measure judge agreement. With paired scoring and a per-item SD near 1.0 to 1.5, about 90 to 200 items detect a 0.3 rubric change at 80% power, which 60 cannot. Pre-register the thresholds.

Phase C: Training v4. Move the notebook to Unsloth (halves run time, lifts the token cap so the 55 dropped chart rows return). Spend 3 GPU hours on 200-step smoke tests of three bases, then pick by quiz and chart-claim rate: Qwen3-8B QLoRA (known quantity), Qwen3.5-4B in plain fp16 LoRA (fits the T4 without quantisation, avoids the merge trap, leaves room for GRPO, and scores above Qwen3-4B-2507 on independent indices), Qwen3.5-9B QLoRA (strongest, but a new hybrid architecture whose bitsandbytes and llama.cpp support must be confirmed first). Settings: r=32, alpha=64, rsLoRA on, dropout 0, weight decay 0.01, all seven projections, 2 epochs, effective batch 8, max length 4,096, learning-rate sweep over 2e-4, 5e-4 and 1e-3 for 300 steps each before the full run. Add a callback that greedy-decodes 100 quiz items every 200 steps so checkpoints are chosen on the metric that matters, not validation loss. Thinking: keep the code-written working for chart rows (about 30% of rows) and mask the empty think block for the rest.

Gate C: quiz 85 or more, rule fidelity 3.3 or more, safety 95% or more, every headline metric better than v2-8b with a confidence interval that excludes zero.

Phase D: Verifier loop. Sample 4 answers per chart prompt at T=0.7 from the v4 model, score them with the chart-claim checker and the judge, keep the best verified answer, and fine-tune one more epoch on those (rejection-sampling fine-tuning; most of the gain comes in round one). Build 2,000 preference pairs where chosen = verified, rejected = contradicts the chart or invents a book, and run ORPO or DPO (Tian et al. cut factual errors 40% to 58% this way). If a 4B base was chosen, run GRPO with a programmatic reward (claim consistency plus mild format and length terms, never a harsh penalty) on the T4; with an 8B base rent an L4 (RunPod lists $0.44 to $0.59 per hour, so 20 hours is about $12).

Gate D: 95% of chart claims verify, rubric mean 4.0, rule fidelity 3.8, no safety regression.

Phase E: Serving facts and guards. Put a deterministic lookup block into the prompt for concept and placement questions (sign lords, dignities, nakshatra lords, aspects, yoga definitions from the engine's tables). This is not retrieval of passages and does not breach the constitution's no-RAG rule, but the wording of Principle I.3 and section 4 should be clarified in a v1.3 revision. Add an output recount of every chart claim, regenerating once on failure. Add Qwen3Guard-0.6B (Apache 2.0, 119 languages) as a learned input and output classifier beside the regex layer. Test runtime LoRA loading in llama.cpp against the fp16 merge on the full eval and ship whichever scores higher.

Phase F: Release readiness is listed in section 9.

Compute and cost

Phase	GPU hours	Where	Cash cost
A Data v4	0	laptop, Claude and Gemini subscriptions	subscription time only
B Evaluation v2	3 (base-model answers on the new sets)	Kaggle T4	0
C Training v4	30 (3 smoke, 6 sweep, 20 full run, 1 eval)	Kaggle T4, two accounts	0
D Verifier loop	20	Kaggle T4, or L4 for GRPO on 8B	0 to about $12
E Serving tests	5	Kaggle T4	0
Total	about 60		under $20

Two bars should be redefined, not relaxed. Safety at 100% as judged by a single LLM is not a measurable target: the two judges already disagree on 6 of 50 items. Propose: 100% on the hard rules (no death or timing prediction, no diagnosis, no trading instructions, helpline on self-harm, no verse citations) enforced and measured by code and the classifier, plus 95% or better on the judged rubric under both judges. Likewise rule fidelity 3.8 should be measured against the reference answer with reference-guided grading, which Zheng et al. show cuts judge error from 70% to 15% on checkable content.

What this will and will not achieve. Phases A to D are the evidence-backed route to the release bar, and the v2 fact rows show the mechanism works on this exact model and data. The residual risk is that an 8B model will still invent rules on long open-ended answers; Phase E's lookup block and recount are the hedge, and if Gate D is missed after one verifier round the honest conclusion is that this product needs retrieval of passages at inference, which would be a constitution change rather than another training run. Hindi output stays in the backlog: Qwen3's tokenizer spends about 4.8 tokens per Devanagari word against 1.3 for English, so Hindi answers would be nearly four times as expensive to train and serve.

Serving, safety and monitoring for production

A model that passes Gate D is still not a product. The items below are what a public deployment needs on top, most of them cheap and none of them GPU-bound.

Serving stack.

- Keep llama.cpp `llama-server`: it already offers an OpenAI-compatible API, continuous batching, parallel slots and runtime LoRA adapters (server docs). Serve Q8_0 on a GPU host (quality within noise of fp16 in this project's own A/B/C chain) and keep Q4_K_M only for the laptop. Measured here: Q8_0 writes 28 tok/s and reads about 1,150 tok/s on one T4; Q4_K_M 30 tok/s. A 2,500-token chart prompt therefore costs about 2.3 s before the first token, and a 600-token answer about 20 s. Use `cache_prompt` so follow-up questions on the same chart reuse the prefix.
- If concurrency matters more than cost, vLLM with Qwen3-8B on one 24 GB A10 reaches about 100 tok/s aggregate at 4 users and 850 at 64 (TrueFoundry benchmark); vLLM also serves LoRA adapters unmerged. The Hugging Face Docker Space in `deploy/hf_space/` is ready but needs a paid plan to run.
- Pin everything in the export manifest: llama.cpp commit (one export has "not recorded"), quantisation type, adapter hash, system-prompt hash, render version. Serve the chart engine as the same Python package version used to build the training data.

Safety architecture (defence in depth).

1. Deterministic input rules (exist) for the categories that must never reach the model, plus the two added after the robustness run (system-prompt extraction, caste ranking). Treat the regex set as a compiled policy file with tests, and re-run the 2,704-question over-blocking check on every change, as was done once.
2. A learned classifier on input and output. Qwen3Guard 0.6B (paper, Apache 2.0, 119 languages, three-tier safe/controversial/unsafe) is the natural fit for a Qwen stack and covers Hindi; Llama Guard 4 is stronger but carries the Llama licence. The current scipy classifier was trained on the project's own prompts and should become a fallback only.
3. Output verification that is specific to this domain: recount every house, lordship, dignity and "Nth from X" claim against the chart dict; block verse citations and book titles not in the corpus list; regenerate once with the failure named, then fall back to the fixed reply. This replaces the two regex output rules that produced false positives.
4. Policy text. Write the safety policy as a document (what is refused, what is redirected, what wording is required, which helplines by country) and derive the eval prompts, the fixed replies and the guard thresholds from it, so a change in one place propagates.

Red-teaming before launch. Follow the NIST Generative AI Profile's pre-deployment red-team guidance (NIST AI 600-1) and test explicitly for OWASP LLM01 prompt injection and LLM07 system-prompt leakage (OWASP 2025), both of which the robustness set already caught once. Write the prompts before the next training run, keep them out of training, and log every finding with its fix.

Evaluation in production. Run the full Gate D suite on every candidate artifact (adapter, merged, quantised) and refuse promotion on any safety regression, as Principle IX says. Add a weekly sample of 50 live conversations (consented, birth data stripped as `server.py` already does) rated by the two judges and spot-checked by a person; track the chart-claim verification rate, refusal rate by category, over-refusal rate, and judge agreement over time.

Monitoring and operations. Structured JSONL traces exist; add per-request latency, token counts, guard route, verifier result and model/adapter hashes, and alert on refusal-rate drift, verifier-failure spikes, latency p95 and any output-rule block. Keep a rollback to the previous adapter one flag away. Load-test to the concurrency you expect before publishing a link.

Licensing and privacy, blocking items. Review the copyright status of the 46 source books and the terms of the teacher models before any weights or data leave the private repo (Principle VI; task 0.7 has been open since the start). Confirm no real birth data entered training (synthetic births only, already the case) and state the retention policy for live birth data in the UI. Show the disclaimer on every answer, not only on the first.

Documentation debt to clear. README with usage and limits, the Ollama Modelfile, the hand check of 10 charts against Jagannatha Hora, and this report's decisions folded into `STATUS.md` and a constitution v1.3 (deterministic lookups allowed; safety bar redefined as hard rules by code plus judged rubric).